

CodePath AI301 — Open Source Capstone

Assignment 1: Issue Selection

Step-by-Step Guide and Submission Skeleton

Deliverables at a glance (25 points)

Where in ai301-coursework	What	Points
tools/issue-select/	rubric.md, SKILL.md, scope.md, references/ evidence-guide.md	6
beat-1-sandbox/unit-1/eval-run.txt	one complete harness run (--save-run)	3
beat-1-sandbox/unit-1/selection.md	four write-up fields	7
beat-1-sandbox/unit-1/selection.md	chosen issue, verdict, reflection	9

Submit the link to the **whole repo**, not a folder.

1 Create the course repo

- 1.1 Open the course template on GitHub → **Use this template**.
- 1.2 Name it ai301-coursework, set it to **Public**, create it.
- 1.3 Confirm that tools/issue-select/ and beat-1-sandbox/unit-1/ exist (or will be created when you upload).

2 Install the skill

On Windows, run these in **Git Bash**, where ~ is C:\Users\Luis Mendez.

```
mkdir -p ~/.claude/skills
cp -r path/to/unit-1-materials/skill ~/.claude/skills/issue-select
ls ~/.claude/skills/issue-select
# expect: SKILL.md scope.md rubric.md references/evidence-guide.md
```

Edit ~/.claude/skills/issue-select/scope.md and set its repo line to:

```
Repo: codepath/pathreview-ai301-fa26-s1
```

One rubric, two uses

The installed rubric.md is the *only* copy you edit. Point the eval harness's --rubric at it, and live mode reads it too, so the two always agree.

3 Read before you write

- 3.1 Read `SKILL.md` (how the skill runs and what it outputs), `scope.md` (house rules and the repo), and `references/evidence-guide.md` (what counts as evidence).
- 3.2 Read the empty `rubric.md` and keep its *exact* section layout. The harness and the skill parse it.
- 3.3 Re-read the 4 calibration issues from the activity. They are not scored, so you can use them to sanity-check your checks.

4 Draft the rubric

The motto is “*evidence sources and thresholds, not adjectives.*” A check like “the issue is well scoped” is an adjective. A check that names *where to look* and *what number or condition flips the verdict* is a threshold.

Pattern for one check

Adapt this to the template’s format. Fill in the brackets yourself.

```
### Check [N]: [short name]
- Evidence source: [where the skill looks: issue body, labels, comments,
  assignees, linked PRs, files named in the issue, ...]
- Threshold: [the concrete condition, e.g. a count, a presence/absence,
  a time window]
- Verdict effect: [reject | lowers fit]
- Why: [one sentence: what goes wrong for a first-time contributor if
  this check is skipped]
```

Questions that lead to checks

Answer each one with an evidence source and a threshold:

- Is someone already working on it? Where would you see that, and how recent must it be to count?
- Is the problem reproducible from what is written? What must the issue contain?
- Is it small enough for a first issue? What signals size (files touched, subsystems, open questions)?
- Is it in scope for Path Review’s house rules (see `scope.md`)?
- Does it need access, secrets, or maintainer decisions you cannot get?

5 Evaluate with the cheap loop

From the `eval/` directory of the Unit 1 materials (use `python` or `py` if `python3` is missing on Windows):

```
# full run, about $4
python3 run_eval.py --rubric ~/.claude/skills/issue-select/rubric.md

# re-grade only the disagreements, about $0.20 per issue
python3 run_eval.py --rubric ~/.claude/skills/issue-select/rubric.md \
  --only issue-07,issue-12
```

5.1 After each full run, read the **per-category tallies**. You must match at least one verdict in *every* category.

5.2 For each disagreement, decide: is it one of the “clear” 16 (fix the rubric) or an arguable scope call (maybe leave it)?

5.3 Change *one* check at a time, then use `--only` on the issues it should affect.

5.4 **Log every run as you go.** Your “Run history” field is graded on order, so keep this table:

#	Type	Score	Categories OK?	What I changed before this run
1	full	--/20		initial rubric
2	only			
3				
4				

The bar

Aim for **18/20** with the category floor met. A first run that already scores 20/20 is fine and loses nothing.

6 Record the final run

Once the rubric is final, do one **full** run with `--save-run`:

```
python3 run_eval.py --rubric ~/.claude/skills/issue-select/rubric.md \
  --save-run eval-run.txt
```

Watch out

Never hand-edit `eval-run.txt`. Its header fingerprints your files, so after this run do **not** change `rubric.md`, or the file you upload will no longer match the run. `--only` and `--limit` runs refuse to save.

7 Choose your issue (live mode)

7.1 Browse the open issues in `codepath/pathreview-ai301-fa26-s1` and pick 2–3 candidates.

7.2 Run the skill from any directory:

```
claude "issue-select: grade these candidate first issues: <URL1> <URL2> <URL3>"
```

7.3 Check the output: a ranked read-out (accepted issues with a fit reason, then rejected ones with the check that sank them) and a fenced JSON block with `accept/reject`. **No JSON block means the skill did not run.**

7.4 Of the accepted issues, choose the one that fits you best. Save the verdict output.

7.5 Do not comment on or claim the issue. That happens in Unit 2.

8 Upload and submit

8.1 In `tools/issue-select/`: **Add file** → **Upload files**, then upload the *current* installed `SKILL.md`, `scope.md`, `rubric.md`, and `evidence-guide.md` (inside a `references/` folder).

8.2 In `beat-1-sandbox/unit-1/`: upload `eval-run.txt` and the filled `selection.md`.

8.3 Open the repo in a private browser window to confirm it is public, then submit the **repo** link in the portal.

Skeleton for selection.md

Use the headings that are already in the template file if they differ. Everything in [...] is yours to write, in your own words.

```
## Chosen issue
Link: [https://github.com/codepath/pathreview-ai301-fa26-s1/issues/N]

Skill verdict:
[paste the ranked read-out line and the JSON entry for this issue]

## Reflection prompts
[answer each prompt from the template here]

## Run history
[In order: run 1 (full, score, category tallies) -> what you changed
-> run 2 (--only issue-xx, result) -> ... -> final saved run and score.]

## Issue analysis
Issue: [issue-NN]
My rubric's verdict: [accept/reject]   Gold label: [accept/reject]
[Why the rubric read it that way: which check fired or didn't fire, and
what evidence in the issue it looked at.]

## Check rationale
Quoted check (current wording, identical to the uploaded rubric.md):
> [paste exactly]
[Why this evidence source and this threshold, and what you would lose
with a looser or stricter threshold.]

## Trade-offs
[What your rubric deliberately accepts or rejects on the arguable
calls, and the cost of that choice.]
```

Final checklist

- ☐ Repo ai301-coursework is public and made from the template
- ☐ Skill installed; scope.md Repo line set
- ☐ $\geq 18/20$ and category floor met (or the shortfall explained in the write-up)
- ☐ eval-run.txt from a full --save-run, unedited, and the rubric unchanged since
- ☐ Skill files uploaded to tools/issue-select/
- ☐ selection.md: issue link, verdict, reflection, and all four write-up fields
- ☐ No comment posted on the chosen issue
- ☐ Whole-repo link submitted